```
function addThings(a: number, b: number, c?: number) {
  return a + b
}

const result = addThings(2, 3)
```

In this case, we've specified the `c` argument as optional, meaning that we can call `addThings` with only two arguments.

However, the optional argument is set to the value `undefined` if not used, and that might not be the most helpful. So TypeScript allows for a default value for an argument using basically the same syntax as Ruby:

```
function addThings(a: number, b: number, c: number = 0) {
  return a + b + c
}

const result = addThings(2, 3)
```

In this example, we can now safely add all three arguments because if the third argument is not specified, the default value takes over and `c` is set to `0`. If you leave the type annotation off the default argument, `function addThings(a: number, b: number, c = 0)`, then TypeScript uses inference to determine the type.

Optional arguments need to come at the end of the list of arguments, but arguments with default values can come anywhere in the list. Somewhat awkwardly, you can then trigger the default behavior by explicitly passing `undefined` to that argument.

Sometimes you legitimately want to allow an arbitrary number of arguments to a function, which you can do in TypeScript with the spread operator, `...`:

```
function addThings(a: number, ...others: number[]) {
  return a + others.reduce((sum, value) => sum + value)
}

const result = addThings(2, 3, 4, 5, 6)
```

This time, any arguments passed to the method after the first one are accumulated into the `others` argument, which has an array type. We can then use